

mlxsw tc offloads

netdev 1.2

yotamg

Mellanox TECHNOLOGIES

Connect. Accelerate. Outperform.™

Introduce matchall cls

- Matches everything
- Allows to mirror ports easily:
  - tc filter add dev sw1p5 parent ffff: \
    matchall skip_sw \
    action mirred egress mirror \
    dev sw1p6
- Enable offloading
- skip_sw, skip_hw
- Make mlxsw offload matchall-mirred tc rules
  - spectrum supports mirroring inside hw

© 2016 Mellanox Technologies

Introduce sample action

- Enables sampling traffic that matches classifier
  - Peeks packets randomly with given probability
  - Duplicate the packets
  - Truncates them if asked
  - Marks them and re-injects them to ingress qdisc of device
- Add informative metadata on packet using ife encapsulation
  - orig_size, ifindex
- Example:
  - tc filter add dev sw1p5 parent ffff: matchall \
    action sample rate 12 mark 12 trunc 12 dst <MAC> type 0xdead
- Make mlxsw offload matchall-sample tc rules
  - spectrum supports packet sampling inside hw

© 2016 Mellanox Technologies

ACL Offloads

switchdev

What do we want to enable?

- Multiple Match-Action-Tables
  - Having multiple apps
    - firewall + load balancer + nfv
  - openflow
- Shared qdiscs might be comfortable
- Offloadable
  - ebpf does not count

© 2016 Mellanox Technologies

Option 1

- Flower classifier
- Aggregate rules of the formula
  - tc filter add dev eth1 parent ffff: flower match [...] skip_sw action [...]
- Actions can be one of the supported actions
  - mlxsw hardware supports all flower keys and many tc actions
- Can be ingress/egress and on router ports
- Add support for flow control actions, like continue, reclassify etc.
- Several ACL tables will get compiled into one big table

© 2016 Mellanox Technologies

Option 1

- Pros:
  - Easy to implement as offload support exists in kernel
  - Examples can be found on mlx4/5 NIC driver
- Cons:
  - Terrible for hardware
    - Hardware TCAM can implement several tables

© 2016 Mellanox Technologies

Option 2

- Having multiple table support for tc
- Basic design might be:
  - Having classful qdisc, where each class represent a table
  - Implement jump command
    - Jump to index is currently not supported
    - allow support for jump between classes
- Add support for classful ingress qdiscs
  - Currently not supported

© 2016 Mellanox Technologies

Option 2

- Pros:
  - Can utilize hardware capability
- Cons:
  - Does it fit tc design?

© 2016 Mellanox Technologies

P4?

- A language designed to define network flows
- In that language, the user can define a table, its keys and possible actions
- The rules in that table can only match the table keys and act upon the table actions
- The table can be bound to ingress/egress
- Allows binding more than one table to a port
- Allows aggregating tables
- P4 does not define the API for adding rules to that table
  - Pretty much straightforward when the table keys and actions are known

© 2016 Mellanox Technologies

P4?

- Pros:
  - Multiple table support is inherent
  - P4 is getting more and more popular among the user base
  - Enables full utilization of hardware
- Cons:
  - Might be abused to bypass kernel networking
    - so does ebpf
  - Does it fit tc design?

© 2016 Mellanox Technologies

Proposed P4 Arch

- tc p4_define
  - Shared qdiscs might be helpful
- tc p4_add_rule

© 2016 Mellanox Technologies



Thank You

Mellanox TECHNOLOGIES

Connect. Accelerate. Outperform.™